

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Agent Review Reference

Agent Review uses Claude Code to comprehensively evaluate Terminal-Bench tasks for compliance with format requirements, best practices, and quality standards. This automated review provides detailed feedback on task structure, correctness, and potential issues.

Note: Agent Review currently does not block task submission. It is provided as an additional tool to help identify potential issues, warnings, and areas for improvement.

How It Works

Agent Review is an automated process that uses Claude Code with a specialized skill to systematically examine your task directory. It performs static analysis—reading and evaluating files without executing any code or building containers.

The review produces a structured report with issues categorized by severity and actionable fix recommendations.

What Gets Reviewed

Agent Review performs a systematic static analysis of your task, examining all key files without executing any code.

Step 1: File Structure Verification

Checks that all required files exist.

Non-milestone tasks:

- `environment/Dockerfile`
- `instruction.md`
- `task.toml`
- `solution/solve.sh`
- `tests/test_outputs.py`
- `tests/test.sh`

Milestone tasks:

- `environment/Dockerfile`
- `task.toml` (with one `[[steps]]` block per milestone)
- For each milestone `N`: `steps/milestone_N/instruction.md`, `steps/milestone_N/tests/test.sh`, `steps/milestone_N/tests/test_mN.py`, `steps/milestone_N/solution/solve.sh`, `steps/milestone_N/solution/solveN.sh`
- See [Milestones page](#) for details

For multi-container tasks, also verifies:

- `environment/docker-compose.yaml`
- Additional Dockerfiles (e.g., `Dockerfile.server`)
- Required flags in `task.toml`

Step 2: task.toml Configuration

Validates your task metadata:

FIELD	REQUIREMENT
<code>version</code>	Must be "2.0"
<code>author_name</code>	Required. Can be "anonymous".
<code>author_email</code>	Required. Can be "anonymous".

FIELD	REQUIREMENT
<code>difficulty</code>	Must be: easy, medium, hard, or unknown
<code>subcategories</code>	Must be any of (can be multiple): "long_context", "to
<code>codebase_size</code>	Must be <code>minimal</code> , <code>small</code> , or <code>large</code> : rough scale
<code>number_of_milestones</code>	Must be: integer. Must be 0 if no milestones. For m
<code>category</code>	Must be a valid category
<code>tags</code>	Array of relevant tags
<code>[verifier].timeout_sec</code>	Required for non-milestone tasks. Replaced by per-
<code>[agent].timeout_sec</code>	Required for non-milestone tasks. Replaced by per-
<code>[environment]</code>	CPU, memory, storage limits, plus <code>workdir</code>
<code>[[steps]]</code>	Required for milestone tasks. One block per milesto

Step 3: Instruction Quality

Evaluates whether `instruction.md`:

- Provides clear, unambiguous task description
- States success criteria explicitly
- ~~Specifies output format requirements~~
- Includes necessary context

Step 4: Dockerfile Review

Checks `environment/Dockerfile` for:

- Proper `WORKDIR /app` setup

- Correct data file copying
- Dependency installation
- **Anti-pattern detection:** Tests or solution copied to image

Step 5: Oracle Solution Review

Evaluates `solution/solve.sh` for:

- Proper shebang (`#!/usr/bin/env bash`)
- Error handling (`set -euo pipefail`)
- Solution completeness and correctness
- Code clarity and maintainability

Step 6: Test Coverage

Analyzes `tests/test_outputs.py` for:

- Use of Python pytest framework
- Multiple test functions
- Clear, specific assertions
- Helpful error messages
- Edge case coverage

Step 7: Test Runner Validation

Verifies `tests/test.sh`:

- Has proper shebang
- Runs the Python pytest verifier with pinned, pre-installed dependencies
- Does not install packages or download from the network at runtime
- **Writes reward to** `/logs/verifier/reward.txt` (critical)
- Handles working directory validation

Step 8: Advanced Quality Checks

Beyond structure, the review evaluates:

Behavior Coverage

- All tested behavior is described in instructions
- All described behavior is verified by tests

Anti-Cheating Measures

- Can the agent bypass the intended solution?
- Are there shortcuts via data file editing?
- Are solutions findable in environment files?

Structured Data Schema

- JSON/API outputs have exact schema definitions
- Field names, types, and structure are specified

Dependency Pinning

- Python packages have exact version pins
- Every Docker `FROM` image and pulled compose `image:` is digest-pinned with `@sha256:<digest>`
- The final runtime base image is sanctioned or explicitly exempt
- Large optional assets are not baked into `environment/`

Naming and Typos

- File paths are correct
- Variable names are consistent
- API endpoints match between instruction, solution, and tests

Quality Control Criteria

The review enforces these critical guidelines:

No Latency-Based Tests

Tests must not check for performance thresholds—these vary by hardware and aren't reproducible.

Identical Oracle/Agent Testing

Testing logic must be exactly the same for both oracle and agent. Conditional behavior based on execution mode is banned.

Multi-Container Tagging

Tasks with `docker-compose.yaml` must set `is_multi_container = true` and `custom_docker_compose = true` in `task.toml`.

No Web Data Fetching

Tasks should not fetch data from URLs (except package managers). Pre-download data into `environment/`.

Reserved Directories

Dockerfile must not create or modify `/tests`, `/solution`, or `/oracle` directories.

Reward File Required

`test.sh` must always write to `/logs/verifier/reward.txt` after tests complete.

Environment Variable Defaults

When using `$TEST_DIR`, provide a default value. Hardcoded paths and standard variables (`$HOME`, `$PWD`) are fine without defaults.

Issue Severity Levels

Critical (Must Fix)

Issues that make the task invalid or unfair:

- Missing required files
- Invalid TOML syntax or missing fields
- No test functions
- Reward file not written
- Behavior mismatch between instructions and tests
- Tests or solution copied to image
- Hardcoded solution outputs
- Unpinned Python dependencies or base / compose images without digest pins
- Unsansctioned final runtime base image without an exemption
- Oversized `environment/` build context
- Critical path/variable typos
- Missing structured data schema
- Latency-based tests
- Oracle/Agent conditional logic
- Web data fetching
- Reserved directory creation

Warning (Should Fix)

Best practice violations:

- Missing shebang
- Brief or unclear instructions
- Missing recommended fields
- Unreasonable timeouts
- Missing WORKDIR in Dockerfile
- Poor test docstrings
- Test dependency installs in `test.sh`
- Verifier dependencies missing from Dockerfile
- Minor typos
- Questionable task quality for stated difficulty

💡 Suggestion (Optional)

Improvements to consider:

- Code style enhancements
- Additional test coverage
- Documentation improvements
- Better naming conventions

Review Report Format

The review produces a structured report:

📄 Copy

```
### Review Report: [task-name]

**Status:** ✅ PASS | ⚠️ WARNING | ❌ FAIL

**Task Location:** /path/to/task

---

#### Summary
[Brief overview of the task]

---

#### Critical Issues ❌
[Issues that must be fixed, with file locations and suggested fixes]

---

#### Warnings ⚠️
[Best practice violations with recommendations]

---

#### Suggestions 💡
```


[Optional improvements]

Overall Assessment

[Summary with key strengths, weaknesses, and recommendation]

****Recommendation:**** READY TO USE | NEEDS FIXES | REQUIRES REVISION

Example: Critical Fix

Here's an example of a critical issue Claude Code identifies and how to fix it:

Issue: Reward file not written after tests run

The `test.sh` script exits after running pytest but before writing the reward file, causing `RewardFileNotFound` errors.

Before (problematic):

BASH

 Copy

```
#!/bin/bash
set -uo pipefail

mkdir -p /logs/verifier

python -m pytest /tests/test_outputs.py -rA

exit_code=$?
echo "Tests completed with exit code: $exit_code"
exit $exit_code # Exits here - reward.txt never written

if [ $? -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

After (fixed):

BASH

 Copy

```
#!/bin/bash
set -uo pipefail

mkdir -p /logs/verifier

python -m pytest /tests/test_outputs.py -rA
rc=$?

if [ "$rc" -eq 0 ]; then
    echo 1 > /logs/verifier/reward.txt
else
    echo 0 > /logs/verifier/reward.txt
fi
```

The fix ensures the reward file is always written after tests complete, which is required by the harness to determine task success or failure.

Acting on Review Feedback

When addressing review findings:

1. **Start with Critical Issues** — These block task validity
2. **Fix Warnings** — Improve task quality and reliability
3. **Consider Suggestions** — Optional but beneficial

Next Steps

- [Reviewer Checklist](#)
- [LLMaJ Checks Reference](#)
- [CI Checks Reference](#)
- [Oracle Agent](#)

- [Submit your task](#)

PREVIOUS

[LLMaJ Checks Reference](#)

NEXT

[Review Guidelines](#)